INDEX

1.PROBLEM STATEMENT:

Manual attendance tracking consumes valuable classroom time and is susceptible to manipulation or inaccuracies. Additionally, biometric systems require physical contact and costly infrastructure.

There is a growing need for a **contactless, automatic, and scalable system** that can record attendance in real-time with minimal setup.

2.OBJECTIVE:

- To design and implement an automated attendance system using IoT principles.

- To utilize the ESP32's built-in Bluetooth and Wi-Fi capabilities for scanning and data transmission.
- To integrate Firebase Realtime Database for secure, cloud-based attendance logging.
- To provide an always-on, low-cost, and reliable attendance tracking solution.
- To develop a system that can scale for multiple users and function 24/7.

3.METHODOLOGY

The proposed system uses the **ESP32 microcontroller** as the core component. When powered on, the ESP32 connects to a Wi-Fi network and continuously scans for nearby Bluetooth devices.

Each student's smartphone acts as a Bluetooth beacon.

The ESP32 identifies these devices by their **unique Bluetooth names or MAC addresses**, and upon matching them with pre-stored student identifiers in Firebase, marks them as “Present” or “Absent”.

The attendance data is then uploaded to the **Firebase Realtime Database**, where it is stored with the current date and timestamp. The data can later be accessed through a web or mobile interface for monitoring and reporting.

4 .SYSTEM ARCHITECTURE

Hardware Components:

- **ESP32 DevKit V1:** Microcontroller with Wi-Fi + Bluetooth.
- **Smartphones:** Student's Bluetooth devices acting as unique identifiers.
- **Power Supply:** USB or power bank for portability.

Software Components:

- **Arduino IDE:** Used for programming and debugging the ESP32.
- **Firebase Realtime Database:** Used for cloud-based data storage and retrieval.
- **Firebase ESP Client Library:** For seamless communication between ESP32 and Firebase.
- **BLE Library:** Enables Bluetooth scanning and device detection.
- **Wi-Fi Connectivity:** Used to upload data from ESP32 to Firebase.

5. WORKFLOW

- **Start**
- Initialize Wi-Fi connection.
- Connect to Firebase Database.
- Perform Bluetooth scan.
- For each detected device:
  - Match device name/MAC with student list in Firebase.
  - If match found → Mark as Present with timestamp.
  - If not found → Skip or mark as Absent.
- Upload attendance record to Firebase.

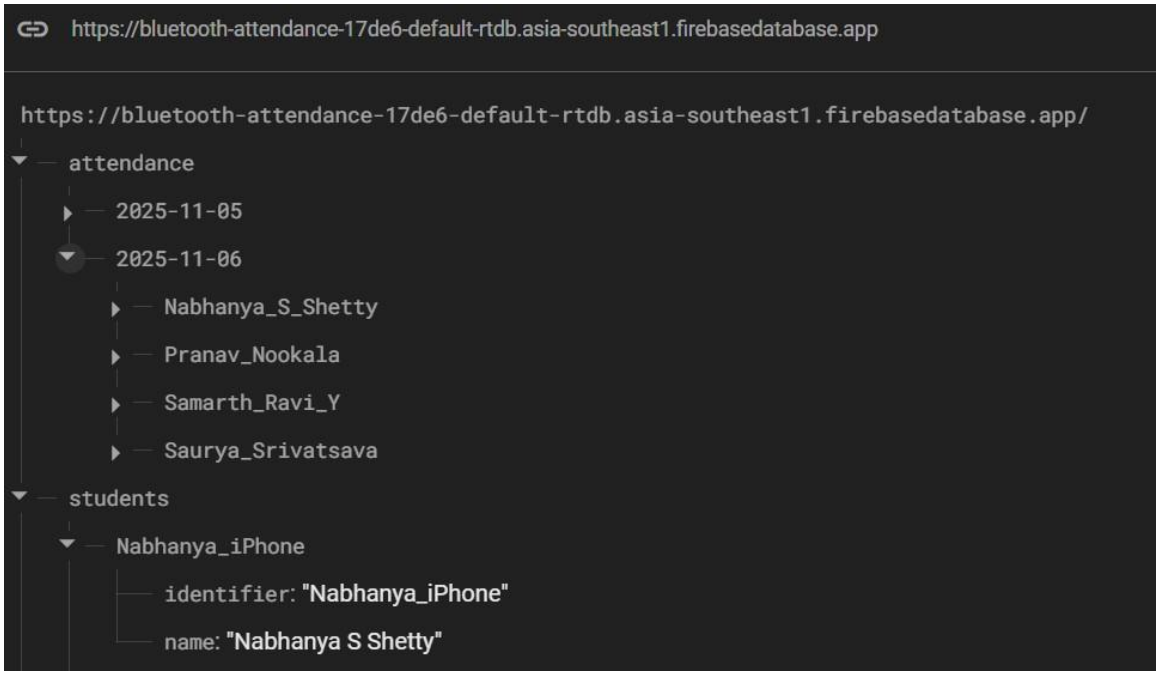
- Repeat scanning at fixed intervals (24/7).
- **End**

6. TOOL/IMPLEMENTATION

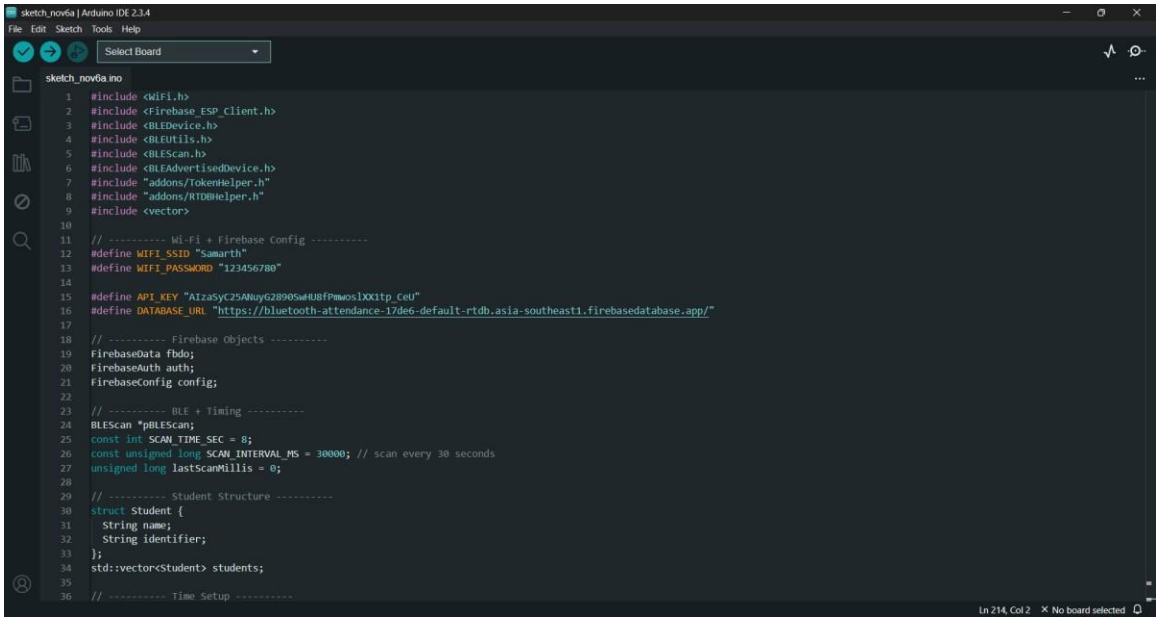
The system is implemented using **Arduino C++** in the Arduino IDE.
Libraries used:

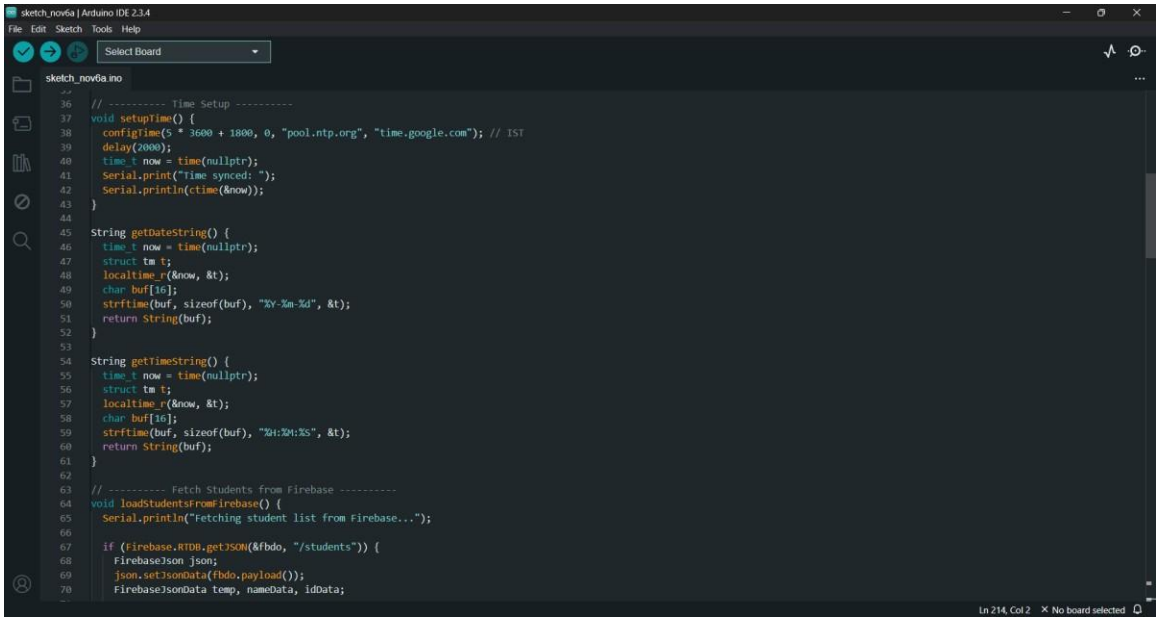
- WiFi.h
- Firebase_ESP_Client.h
- BLEDevice.h
- BLEUtils.h
- BLEScan.h

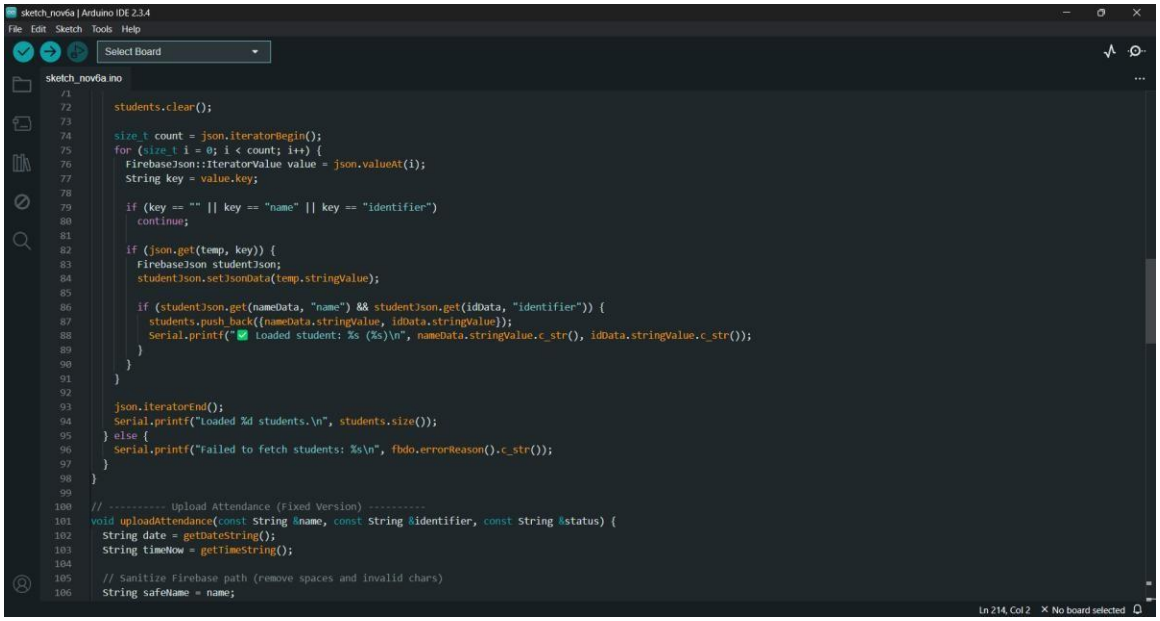
Data is structured in Firebase as:

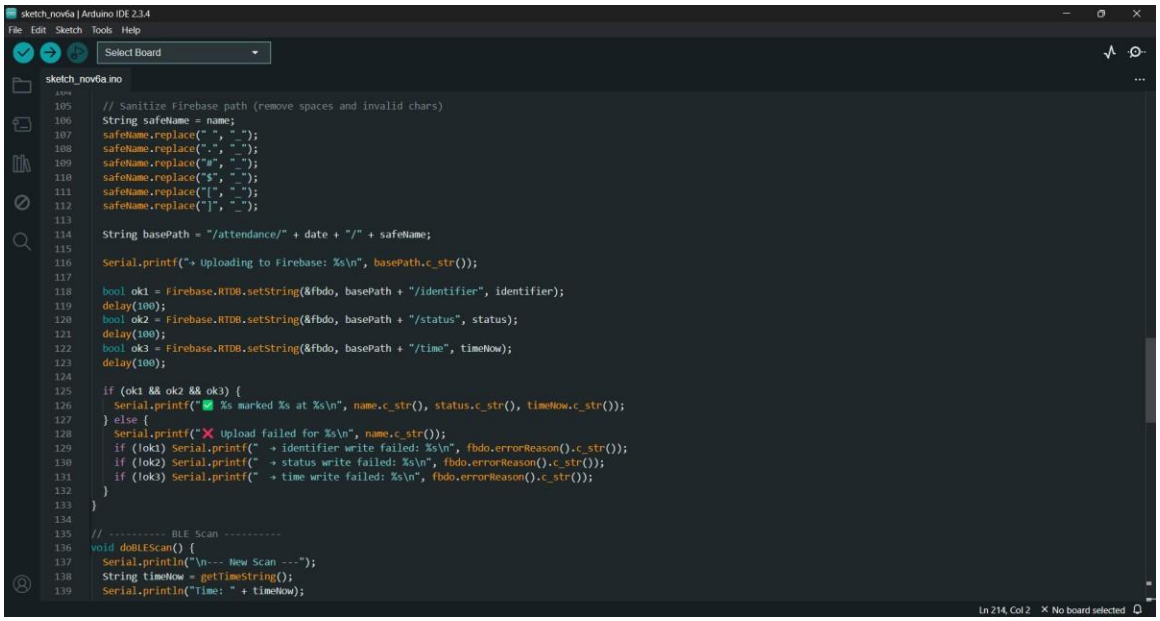


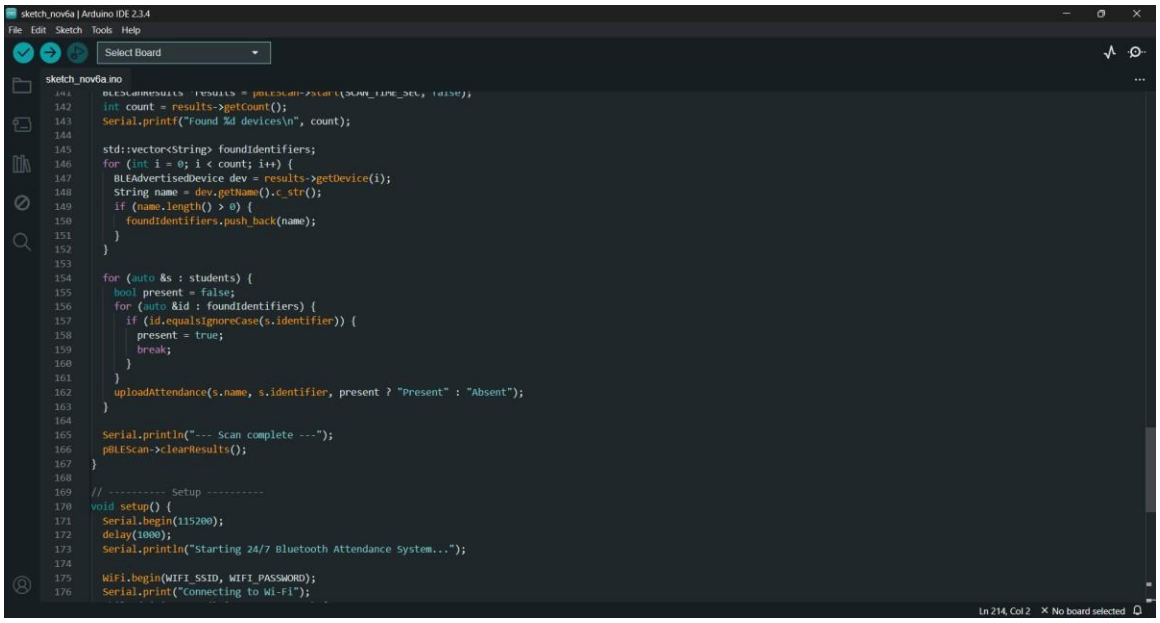
Code for the ESP32

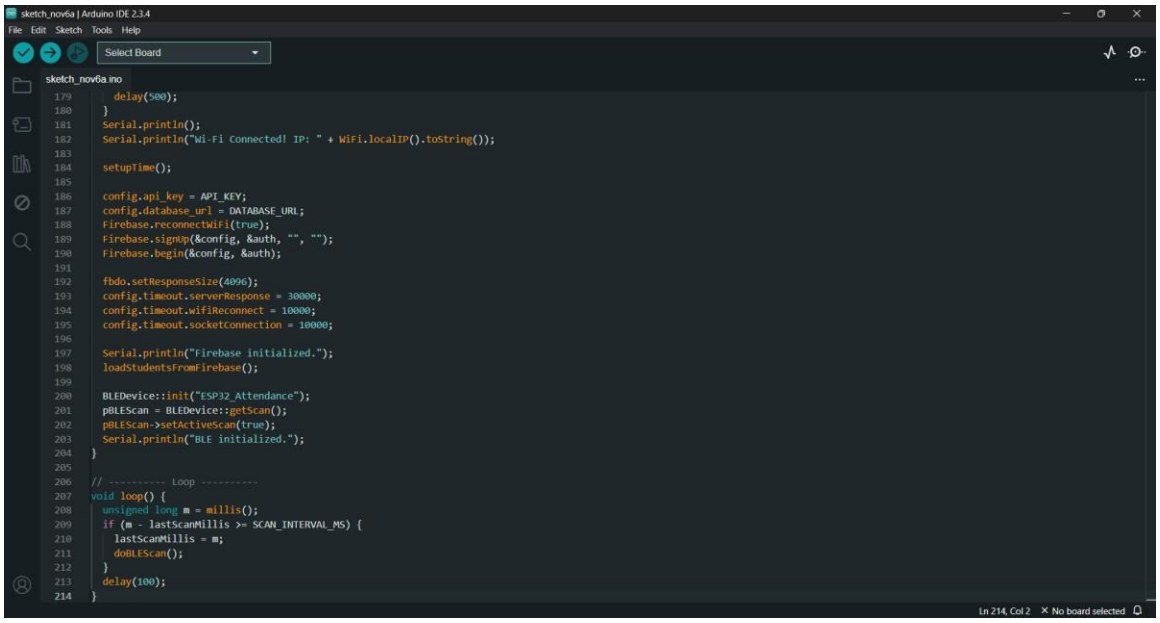

sketch_nov6a | Arduino IDE 2.3.4
File Edit Sketch Tools Help
Select Board
sketch_nov6a.ino
#include <WiFi.h>
#include <Firebase_ESP_Client.h>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>
#include "addons/TokenHelper.h"
#include "addons/RTDBHelper.h"
#include <vector>
// ---------- Wi-Fi + Firebase Config ----------
#define WIFI_SSID "Samarth"
#define WIFI_PASSWORD "123456780"
#define API_KEY "AIzaSyC25ANuyG2890SwHU8fPmwoslXX1tp_CeU"
#define DATABASE_URL "https://bluetooth-attendance-17de6-default-rtdb.asia-southeast1.firebasedatabase.app/"
// ---------- Firebase Objects ----------
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
// ---------- BLE + Timing ----------
BLEScan *pBLEScan;
const int SCAN_TIME_SEC = 8;
const unsigned long SCAN_INTERVAL_MS = 30000; // scan every 30 seconds
unsigned long lastScanMillis = 0;
// ---------- Student Structure ----------
struct Student {
String name;
String identifier;
};
std::vector<Student> students;
// ---------- Time Setup ----------
Ln 214, Col 2
No board selected


sketch_nov6a | Arduino IDE 2.3.4
File Edit Sketch Tools Help
Select Board
sketch_nov6a.ino
// ---------- Time Setup ----------
void setupTime() {
configTime(5 * 3600 + 1800, 0, "pool.ntp.org", "time.google.com"); // IST
delay(2000);
time_t now = time(nullptr);
Serial.print("Time synced: ");
Serial.println(ctime(&now));
}
String getDateString() {
time_t now = time(nullptr);
struct tm t;
localtime_r(&now, &t);
char buf[16];
strftime(buf, sizeof(buf), "%Y-%m-%d", &t);
return String(buf);
}
String getTimeString() {
time_t now = time(nullptr);
struct tm t;
localtime_r(&now, &t);
char buf[16];
strftime(buf, sizeof(buf), "%H:%M:%S", &t);
return String(buf);
}
// ---------- Fetch Students from Firebase ----------
void loadStudentsFromFirebase() {
Serial.println("Fetching student list from Firebase...");
if (Firebase.RTDB.getJSON(&fbdo, "/students")) {
FirebaseJson json;
json.setJsonData(fbdo.payload());
FirebaseJsonData temp, nameData, idData;
Ln 214, Col 2
No board selected


sketch_nov6a | Arduino IDE 2.3.4
File Edit Sketch Tools Help
Select Board
sketch_nov6a.ino
students.clear();
size_t count = json.iteratorBegin();
for (size_t i = 0; i < count; i++) {
FirebaseJson::IteratorValue value = json.valueAt(i);
String key = value.key;
if (key == "" || key == "name" || key == "identifier")
continue;
if (json.get(temp, key)) {
FirebaseJson studentJson;
studentJson.setJsonData(temp.stringValue);
if (studentJson.get(nameData, "name") && studentJson.get(idData, "identifier")) {
students.push_back({nameData.stringValue, idData.stringValue});
Serial.printf("✅ Loaded student: %s (%s)\n", nameData.stringValue.c_str(), idData.stringValue.c_str());
}
}
}
json.iteratorEnd();
Serial.printf("Loaded %d students.\n", students.size());
} else {
Serial.printf("Failed to fetch students: %s\n", fbdo.errorReason().c_str());
}
}
// ---------- Upload Attendance (Fixed Version) ----------
void uploadAttendance(const String &name, const String &identifier, const String &status) {
String date = getDateString();
String timeNow = getTimeString();
// Sanitize Firebase path (remove spaces and invalid chars)
String safeName = name;
Ln 214, Col 2
No board selected

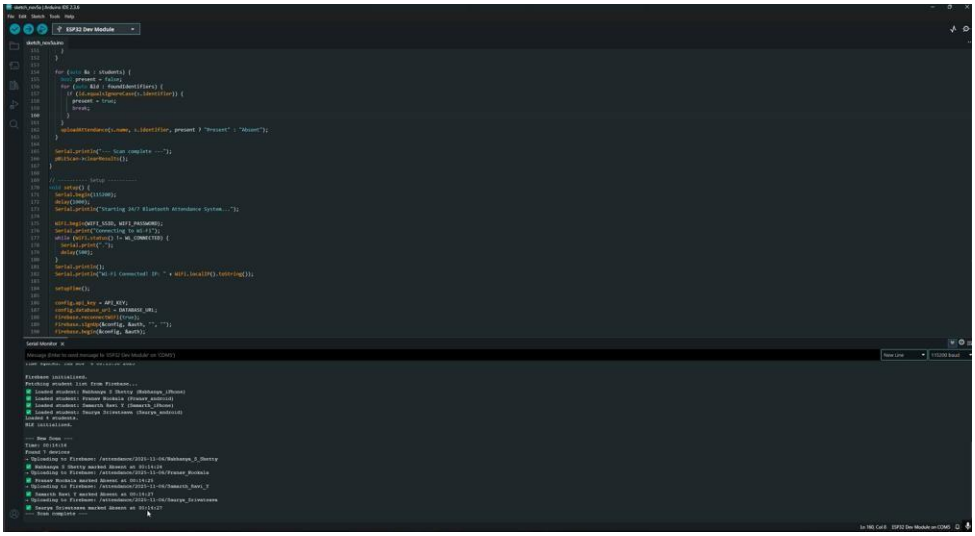

sketch_nov6a | Arduino IDE 2.3.4
File Edit Sketch Tools Help
Select Board
sketch_nov6a.ino
// Sanitize Firebase path (remove spaces and invalid chars)
String safeName = name;
safeName.replace(" ", "_");
safeName.replace(".", "_");
safeName.replace("#", "_");
safeName.replace("$", "_");
safeName.replace("[", "_");
safeName.replace("]", "_");
String basePath = "/attendance/" + date + "/" + safeName;
Serial.printf("→ Uploading to Firebase: %s\n", basePath.c_str());
bool ok1 = Firebase.RTDB.setString(&fbdo, basePath + "/identifier", identifier);
delay(100);
bool ok2 = Firebase.RTDB.setString(&fbdo, basePath + "/status", status);
delay(100);
bool ok3 = Firebase.RTDB.setString(&fbdo, basePath + "/time", timeNow);
delay(100);
if (ok1 && ok2 && ok3) {
Serial.printf("✅ %s marked %s at %s\n", name.c_str(), status.c_str(), timeNow.c_str());
} else {
Serial.printf("❌ Upload failed for %s\n", name.c_str());
if (!ok1) Serial.printf(" → identifier write failed: %s\n", fbdo.errorReason().c_str());
if (!ok2) Serial.printf(" → status write failed: %s\n", fbdo.errorReason().c_str());
if (!ok3) Serial.printf(" → time write failed: %s\n", fbdo.errorReason().c_str());
}
}
// ---------- BLE Scan ----------
void doBLEScan() {
Serial.println("\n--- New Scan ---");
String timeNow = getTimeString();
Serial.println("Time: " + timeNow);
Ln 214, Col 2
No board selected


sketch_nov6a | Arduino IDE 2.3.4
File Edit Sketch Tools Help
Select Board
sketch_nov6a.ino
int count = results->getCount();
Serial.printf("Found %d devices\n", count);
std::vector<String> foundIdentifiers;
for (int i = 0; i < count; i++) {
BLEAdvertisedDevice dev = results->getDevice(i);
String name = dev.getName().c_str();
if (name.length() > 0) {
foundIdentifiers.push_back(name);
}
}
for (auto &s : students) {
bool present = false;
for (auto &id : foundIdentifiers) {
if (id.equalsIgnoreCase(s.identifier)) {
present = true;
break;
}
}
uploadAttendance(s.name, s.identifier, present ? "Present" : "Absent");
}
Serial.println("--- Scan complete ---");
pBLEScan->clearResults();
}
// ---------- Setup ----------
void setup() {
Serial.begin(115200);
delay(1000);
Serial.println("Starting 24/7 Bluetooth Attendance System...");
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
Serial.print("Connecting to Wi-Fi");
Ln 214, Col 2
No board selected

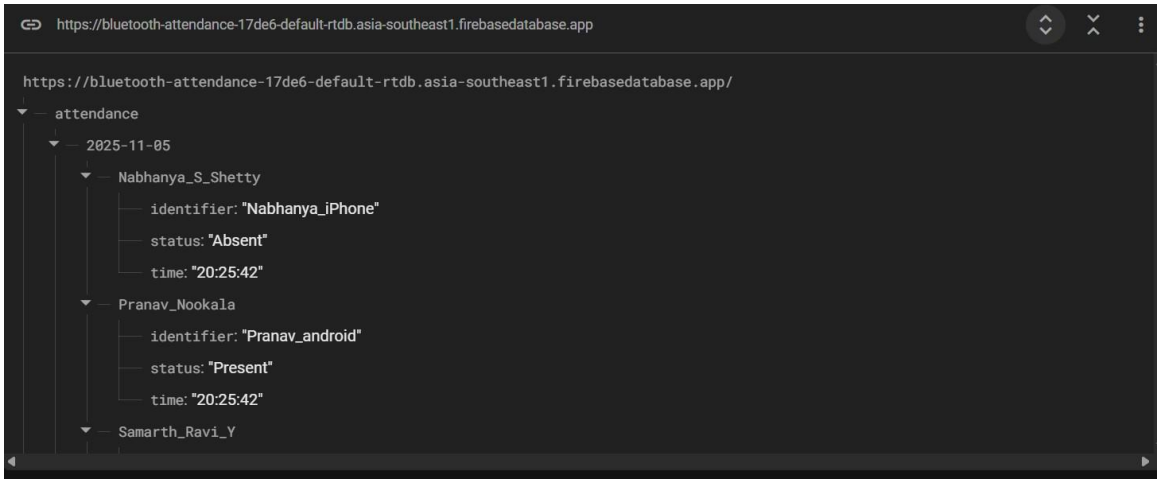

sketch_nov6a | Arduino IDE 2.3.4
File Edit Sketch Tools Help
Select Board
sketch_nov6a.ino
delay(500);
}
Serial.println();
Serial.println("Wi-Fi Connected! IP: " + WiFi.localIP().toString());
setupTime();
config.api_key = API_KEY;
config.database_url = DATABASE_URL;
Firebase.reconnectWiFi(true);
Firebase.signUp(&config, &auth, "", "");
Firebase.begin(&config, &auth);
fbdo.setResponseSize(4096);
config.timeout.serverResponse = 30000;
config.timeout.wifiReconnect = 10000;
config.timeout.socketConnection = 10000;
Serial.println("Firebase initialized.");
loadStudentsFromFirebase();
BLEDevice::init("ESP32_Attendance");
pBLEScan = BLEDevice::getScan();
pBLEScan->setActiveScan(true);
Serial.println("BLE initialized.");
}
// ---------- Loop ----------
void loop() {
unsigned long m = millis();
if (m - lastScanMillis >= SCAN_INTERVAL_MS) {
lastScanMillis = m;
doBLEScan();
}
delay(100);
}
Ln 214, Col 2
No board selected

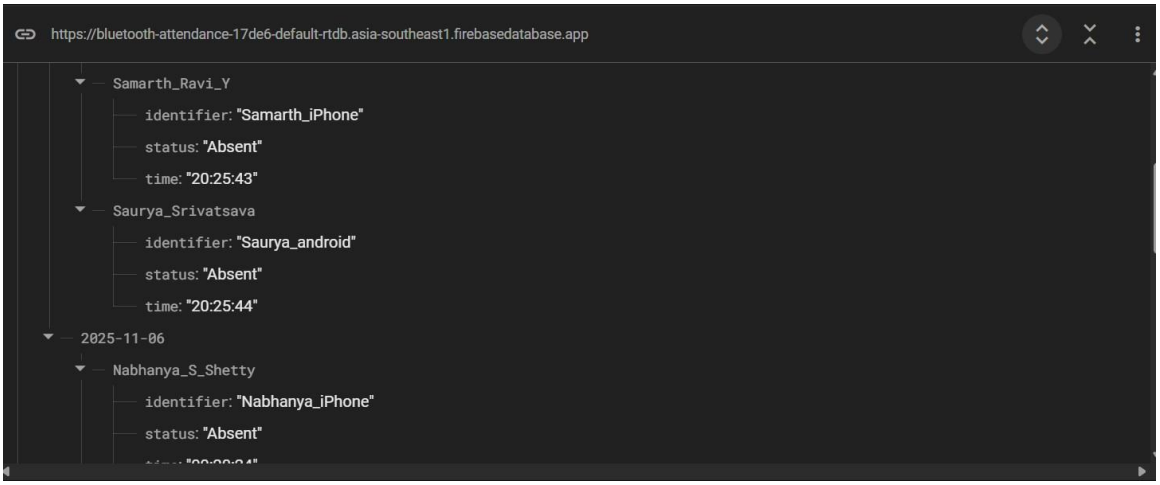
7.VALIDATION or STIMULATION

ESP32 connecting to wifi and scanning for Bluetooth devices



Data uploaded to firebase database by esp32 in real time





Website frontend connected to firebase with live data from esp32

8. RESULTS AND CONCLUSION:

The system successfully detects Bluetooth devices corresponding to registered students and updates their attendance records on Firebase.
The data includes the student's name, device identifier, and timestamp.

Key outcomes:

- Real-time attendance marking verified through the serial monitor.
- Firebase updates confirmed within seconds.
- System operates continuously with minimal power consumption.

- Attendance can be viewed remotely via Firebase console or web interface.

WEBSITE

https://attendance-chi-six.vercel.app/